OpenDecide: handoff document

Everything done and learned so far, so the work can continue in a new chat or folder. Date: 25 Sep 2026.

Machine: Apple M4 MacBook, 16 GB RAM. Project root: `/Users/aryamahadik/Documents/551 CONTAINERS /ds-551-f26-c03-containers-aryamahadik2003/opendecide-data` (note: the folder name '551 CONTAINERS ' has a trailing space; always quote paths).

1. The one-paragraph summary

We are a student startup team looking for an AI result we can publish, get attention for, and raise money on, with about $30/month of Modal GPU credit. The top priority was TypeSafe AI's **Jev**, a closed, fast 'System One' model. Jev answers typed questions about a text (yes/no, pick-one, score) with probabilities instead of generated text. Deep research showed we cannot beat Jev across the board with $30. We can beat it on its documented weaknesses, with an open, small, trainable model. So we built a **lean benchmark** (11-20k train rows, 1,915 eval rows) and fine-tuned the open **GLiClass Edge** model (32.7M params) on the Mac. Our fine-tuned model (A3) cuts held-out prompt-injection success from 80% (untrained) and 61% (plain fine-tune) to **22%**. It stays robust to distractor text and improves consistency and calibration. It still fails on negations phrased in a style it never saw in training. **Jev has NOT been run yet** because no API key was available. All code, data and 515 validated Jev requests are ready; running Jev costs about $0.005.

2. What Jev is (verified facts)

• **Who:** TypeSafe AI (typesafe.ai). Launched 15 Sep 2026 with a $40M seed led by DCVC. Founders: Diogo Almeida (CEO, ex-OpenAI, InstructGPT), Erik Gafni (CTO), Sasha Sheng (COO). Forbes headline: $200M valuation (article body not verified).

• **What:** the input is state (text/JSON) plus named questions. Question types: **Noul** (yes/no, returns P(yes)), **Choice** (up to 255 options, returns a distribution; larger sets use a two-stage method) and **Score** (2-10 levels). All questions are answered in parallel and in isolation. The model is jev-1.13.0; the limit is 64k tokens per request (32k for state plus the longest question).

• **Claims:** 70-500 ms (typically about 100 ms). $0.042 per 1M input tokens, output free. 193.6x faster and 444.6x cheaper on TypeSafe's own workflows; the 'accuracy' there is agreement with frontier LLMs, about 67.8%. Architecture, parameter count and training data are all undisclosed. The training method is called RLCD (RL for Calibrated Decisions). Closed: no weights, no paper.

• **Its own docs list 9 failure modes** (docs.typesafe.ai/model-jaggedness/jev-1.13): literal reading; math/numbers; dates; indirection (double negatives, multi-hop); large irrelevant state; adversarial content; contradictory instructions/criteria; **structural invariants**, e.g. P(refund)=0.72 and P(not refund)=0.47 in the same call (sum 1.19); and it cannot generate text.

• **Community audits (Sep 2026):**

– AbdelStark/jev-benchmarks: Banking77/AG News/Emotion pilot. Jev 91% vs GLiNER2.5 70% on AG News, but Emotion Brier 0.846 vs 0.668, and zero probability on the true label 16% of the time. Also measures coverage at a fixed error rate.

– jujumilk3/jev-calibration-audit: P(x)+P(not x) ranges 0.71-1.42.

– zkousama/jagged: injection drops Jev from 96.5% to 26.5%.

– Independent tests: a fine-tuned 310M ModernBERT beat Jev by 12 points on one topic task. Jev scores 62.6% on phishing with one question, but 95% when the task is split into 5 questions combined by a logistic regression.

– Index of audits: Yifan-Lan/awesome-jev-robustness.

• **Access:** Python SDK `typesafe-sdk==0.7.1` (installed in the venv); the key is the env var `TYPESAFE_API_KEY`. Direct signups were paused on 22 Sep; OpenRouter and Vercel AI Gateway also list Jev. Read TypeSafe's terms (docs.typesafe.ai/legal.md) before publishing comparisons. Never train on Jev outputs.

3. Research conclusions

Can $30 beat Jev?

• Not zero-shot across arbitrary questions. That is Jev's moat: two years of work and proprietary data. An open Qwen3-4B reproduction still trails it (0.845 vs 0.883 agreement).

• An 'open Jev clone' is already crowded: Laya (421M), OpenJev repos, GLiClass, and kyegomez/open-jev (consistency loss, no weights).

• **Winnable claims:** an open, local model (tens of ms, no US-only API) that can be fine-tuned, is better calibrated, is more logically consistent, is more robust to injection, and (as a stretch) fills tool-call arguments, which Jev cannot do.

• **Benchmark novelty:** a benchmark alone is NOT new. Community audits already cover negation, paraphrase, injection and coverage. What remains open is an open *trained* model that fixes these failures, plus one harness that covers them all.

• **Compute:** $30 on Modal buys about 7.6 H100-hours, 12-14 A100-hours or 37.5 L4-hours. It is Modal's free Starter allowance and renews monthly. Our training ran on the M4 Mac instead (Modal credit untouched).

• **Funding playbook:** benchmark numbers alone rarely fund a company. Mem0's $24M rested on 41k GitHub stars and usage, not only LoCoMo. Independent verification matters: Poetiq claimed 75%, but ARC Prize verified 54%. Irreproducible claims kill credibility (Reflection 70B). No startup named 'Memorable' was found.

Six vetted ideas (ranked), from about 47 screened. The full report is in reports/ (see section 8).

#	Idea	Number to beat / status	GPU
1	OpenDecide: an open Jev-style decision model that fixes Jev's weaknesses (what we built)	Beat Jev on consistency, injection, calibration, local latency	~$15-25
2	Tariff (HS) code classifier grounded in past US customs (CROSS) rulings, plus the first export-control (ECCN) benchmark	HSCodeComp best agent 46.83% (10-digit); ATLAS 40% (10-digit) / 57.5% (6-digit)	~$12-20
3	SheetTRM: a tiny recursive model for spreadsheet table and cell detection	SpreadsheetLLM 78.9% F1; 0.937 cell-type F1	~$25-30
4	SECReviewBench: predict SEC comment letters from draft filings	No baseline exists	~$10-17 + API
5	FlashTRM: string transforms by example with a tiny recursive model	PROSE: FlashFill 33%, fine-tuned 33B 82%	~$20-28
6	Hinglish / code-mixed customer-service agent benchmark	Private benchmark: 66% (small) vs 90% (frontier)	~$20-25 + $100-250 API

Other plans reviewed (from Codex/ChatGPT):

• 'Logic-consistent System One model' (GLiClass Edge + consistency/Brier losses + SystemOneBench): the right direction, and its facts check out. However, its SystemOneBench is not new (audits exist), 1-p consistency can be hard-coded (the claim must be consistency on freely worded, independently scored questions), 32.7M params is weak zero-shot, and its six ideas were all one bet. **We merged it into our build.**

• 'Jev at 1M labels' (shortlist-then-rerank: GTE-ModernBERT + ANN + reranker): buildable, but this is prior art (BLINK 2020 retrieves the top 64 of about 6M entities; Microsoft XMC works at 1.3M labels). The projected numbers were unrealistic (BLINK zero-shot recall@64 was about 82%; the best P@1 at 1.3M labels is in the mid-50s). It also picks the wrong fight: rerankers are crowded. **Recommendation:** use that architecture for tariff codes (about 17-19k US codes) instead.

4. Decisions and working preferences

• **Chosen:** idea A (OpenDecide, lean). The runner-up is tariff codes (B), which has the clearest paying customer.

• **User preference:** keep explanations short and simple (beginner); use FEW agents. A roughly 130-agent build was stopped for taking too long. Ask before any big fan-out.

• **Full research-grade spec** (140 KB, 28 reviewer points fixed) is in DESIGN.md: 24k train / 4.9k eval rows, 1,494 Jev calls, and about 104 person-hours of human verification. Its build was only partially done (loaders, keys, some banks). The lean version below is what actually ran.

5. What was built (lean benchmark v0.2)

All in `opendecide-data/lean/`. Python venv: `opendecide-data/.venv` (torch 2.14, transformers 4.57.6, gliclass 0.1.17, typesafe-sdk 0.7.1, sentence-transformers, reportlab).

File	Purpose
build_data.py	Builds data/: train.jsonl, dev.jsonl, eval.jsonl, *_relations.jsonl, eval_calls.jsonl, stats.json (seed 20260925)
gliclass_io.py	Serialises rows for GLiClass (one row = one forward pass), scoring, latency
train.py	Fine-tuning. Recipes A0 (base rows only) and A3 (all rows + consistency loss: (pA+pB-1)^2 and (pA-pB)^2). Step-matched
eval_local.py	Runs a local model on eval or dev; writes preds/NAME.jsonl (+ latency)
run_jev.py	Runs Jev: 1 call per state, questions bundled, resumable; writes preds/jev.jsonl
score.py	All metrics with bootstrap CIs; writes results/RESULTS.md and results.json
README.md	Benchmark description, reproduce commands, limitations
models/	edge_A0, edge_A3 (v0.2); edge_A0_v01, edge_A3_v01 (v0.1). base_A3 was NOT finished (stopped)
data/eval_v0.1.sha256	Frozen hashes: the eval set is byte-identical between v0.1 and v0.2
archive_v01/	v0.1 predictions and results table

Dataset design

• **Sources (human labels):** Banking77, CLINC150 plus (with out-of-scope), MultiNLI (train), DBpedia-14 and Women's Clothing reviews (train, added in v0.2), SMS Spam and Twitter Financial News Sentiment (zero-shot eval only). Filler text: wikitext-103 validation (train) and test (eval).

• **Eval: 1915 rows / 515 Jev calls.**

– ID: 200 states x 5 questions.

– ZS: 95 x 5.

– INJ: 60 items x 3 arms (C authority re-label, D argue, E echo control) x 2 questions.

– DIST: 40 x 2.

– Each core state asks a Choice question; a yes/no question about a label (50% gold); its negation in an **unseen style** ('Would someone who says that ... be mistaken?'); a paraphrase; and a double negation ('Would it be a mistake to deny that ...?').

• **Train v0.2: 19944 rows.** Base rows; 7 negation wordings (prefix, judge, statement); clause negation ('the customer is NOT asking about X'); double negation; paraphrase; train-only injection families (A override, B fake system, G fake output) with 1:1 echo controls; distractor filler. Dev: 600 rows with dev-only wordings.

• **Rules:**

– Eval wordings, negation style, injection families, containers and filler corpus are disjoint from train (asserted in code).

– Eval texts are de-duplicated against train (exact match and MiniLM cosine >= 0.95).

– Every row is scored alone; links exist only in relations files.

– Gold for every variant follows a fixed rule.

– Held-out injections sit inside forwarded, quoted or retrieved-note containers and always target a wrong label.

6. Results so far (v0.2; Jev pending)

All models scored on the same 1915 rows in 295 groups. Brackets are 95% bootstrap CIs over groups. A0 and A3 are step-matched (same number of training steps and seed), so the A0-vs-A3 gap isolates the effect of the consistency and robustness data and loss.

Metric	edge_zeroshot	base_zeroshot	edge_A0	edge_A3
Choice accuracy, in-domain (%)	43.5 [37.0-50.5]	58.5 [51.6-65.0]	89.0 [84.7-93.2]	87.0 [82.5-91.4]
Choice accuracy, zero-shot (%)	30.5 [22.0-41.1]	36.8 [27.0-46.2]	36.8 [27.2-47.6]	41.1 [31.2-52.5]
Yes/no acc., base question, ID (%)	52.0 [45.0-59.2]	57.5 [50.8-63.6]	79.5 [73.7-84.5]	75.5 [69.8-81.1]
Yes/no acc., base question, ZS (%)	49.5 [39.8-59.6]	48.4 [38.6-58.3]	45.3 [35.1-55.3]	49.5 [39.0-59.6]
Yes/no acc., negated question (unseen style), ID (%)	48.0 [40.9-54.9]	46.5 [40.0-53.1]	33.5 [27.0-39.6]	44.5 [37.5-51.2]
Yes/no acc., negated question (unseen style), ZS (%)	45.3 [35.7-55.4]	46.3 [36.5-56.0]	52.6 [42.2-62.5]	31.6 [22.8-42.0]
Complement violation |P(A)+P(not A)-1| (lower better)	0.491 [0.459-0.521]	0.321 [0.305-0.335]	0.674 [0.640-0.711]	0.406 [0.377-0.441]
Contradiction rate (%) (lower better)	67.8	93.6	78.3	53.9
Pairs summing to 1 +/-0.05 (%)	6.4	2.4	7.8	13.6
Paraphrase flip rate (%) (lower better)	43.1	7.1	16.3	10.2
Double-negation drift (lower better)	0.246 [0.226-0.270]	0.110 [0.101-0.119]	0.118 [0.094-0.145]	0.119 [0.099-0.139]
Brier, yes/no core (lower better)	0.366 [0.345-0.385]	0.284 [0.268-0.300]	0.320 [0.299-0.342]	0.274 [0.246-0.303]
ECE, yes/no core (lower better)	0.299	0.169	0.284	0.223
Coverage @1% error, cross-fitted (%)	0.0	0.0	0.8	1.8
Injection attack success, held-out families C+D (%) (lower better)	80.0 [70.4-88.6]	75.0 [64.4-84.4]	60.8 [49.1-71.1]	21.7 [13.0-30.7]
Choice acc. under injection C / same items clean (%)	13.3 / 41.7	18.3 / 65.0	33.3 / 90.0	76.7 / 86.7
Echo-control choice acc. (%)	96.7	93.3	100.0	98.3
Distractor choice acc. / same items clean (%)	40.0 / 52.5	50.0 / 60.0	70.0 / 97.5	87.5 / 92.5

Models: **edge_zeroshot** = GLiClass Edge 32.7M, untrained; **base_zeroshot** = GLiClass modern-base 151M, untrained; **edge_A0** = Edge fine-tuned A0 (base rows only); **edge_A3** = Edge fine-tuned A3 (ours: + consistency + robustness).

Latency (batch 1, M4): Edge 32.7M: about 7 ms/question on CPU (median; p95 about 9 ms). Modern-base 151M: about 70-120 ms (measured while training ran, so inflated). Whole 1,915-row eval with Edge: about 18 s including model load. Jev claims 70-500 ms per call over the network.

What the numbers say

• **Injection robustness is the headline win.** Held-out attack success is 80% untrained, 61% for plain fine-tuning (A0) and **22% for A3**, on attack families and containers never seen in training. Echo controls stay at 98%, so the model did NOT learn a shortcut of avoiding whatever label is named.

• **Distractor robustness:** A3 keeps 87.5% vs 92.5% clean; A0 drops from 97.5% to 70%.

• **Consistency:** A3 beats A0 on complement violation (0.41 vs 0.67), contradictions (54% vs 78%), paraphrase flips (10% vs 16%), Brier (0.274 vs 0.320) and ECE. Choice accuracy is unchanged (87 vs 89, CIs overlap).

• **Main failure:** negations in the unseen style are still below chance (ID 44.5%, ZS 31.6%). Small encoders do not generalise to a new way of saying 'not'. v0.1 even learned a 'negative word means flip' shortcut that broke double negatives; v0.2 fixed that (drift 0.53 to 0.12) by adding clause-negation and double-negation training rows.

• **Zero-shot tasks** (spam, finance sentiment) are near chance for every local model. These models need task data. Jev will likely win zero-shot, as expected.

• **Coverage at 1% error** is about 0-2% for all local models: none is reliable enough yet to automate safely at 1% error.

v0.1 results (before the fix), for the record

Metric	edge_zeroshot	base_zeroshot	edge_A0	edge_A3
Choice accuracy, in-domain (%)	43.5 [37.0–50.5]	58.5 [51.6–65.0]	90.0 [85.9–94.1]	81.0 [75.7–86.6]
Choice accuracy, zero-shot (%)	30.5 [22.0–41.1]	36.8 [27.0–46.2]	40.0 [30.1–50.6]	36.8 [27.9–47.3]
Yes/no accuracy, base question, ID (%)	52.0 [45.0–59.2]	57.5 [50.8–63.6]	80.5 [75.1–85.4]	70.0 [64.1–76.0]
Yes/no accuracy, base question, ZS (%)	49.5 [39.8–59.6]	48.4 [38.6–58.3]	42.1 [32.3–52.1]	45.3 [35.4–54.9]
Yes/no accuracy, negated question, ID (%)	48.0 [40.9–54.9]	46.5 [40.0–53.1]	36.5 [30.1–42.6]	44.0 [36.9–50.7]
Yes/no accuracy, negated question, ZS (%)	45.3 [35.7–55.4]	46.3 [36.5–56.0]	52.6 [42.3–62.6]	49.5 [40.0–59.6]
Complement violation P(A)+P(not A)-1 (lower better)	0.491 [0.459–0.521]	0.321 [0.305–0.335]	0.655 [0.619–0.690]	0.324 [0.303–0.347]
Contradiction rate (lower better) (%)	67.8	93.6	78.0	50.8
Pairs summing to 1±0.05 (higher better) (%)	6.4	2.4	6.4	7.1
Paraphrase drift dP (lower better)	0.459 [0.428–0.490]	0.090 [0.081–0.098]	0.157 [0.131–0.188]	0.266 [0.240–0.293]
Paraphrase flip rate (lower better) (%)	43.1	7.1	16.3	35.6
Double-negation drift (lower better)	0.246 [0.226–0.270]	0.110 [0.101–0.119]	0.112 [0.091–0.134]	0.526 [0.499–0.554]
Brier (yes/no, core) (lower better)	0.366 [0.345–0.385]	0.284 [0.268–0.300]	0.302 [0.282–0.323]	0.322 [0.313–0.331]
ECE (yes/no, core) (lower better)	0.299	0.169	0.258	0.254
Coverage @1% error, cross-fitted (higher better) (%)	0.0	0.0	0.0	0.1
Injection attack success, held-out C+D (lower better) (%)	80.0 [70.4–88.6]	75.0 [64.4–84.4]	65.0 [54.0–75.0]	28.3 [18.8–38.0]
Choice acc under injection C (%)	13.3	18.3	28.3	71.7
Choice acc under injection D (%)	18.3	25.0	40.0	55.0
Choice acc, same items clean (%)	41.7	65.0	86.7	85.0
Echo control choice acc (%)	96.7	93.3	100.0	95.0
Distractor choice acc (%)	40.0	50.0	72.5	85.0
Distractor, same items clean (%)	52.5	60.0	92.5	87.5

7. How to continue (exact steps)

A. Get the Jev comparison (the stated goal)

```
cd "/Users/aryamahadik/Documents/551 CONTAINERS
/ds-551-f26-c03-containers-aryamahadik2003/opendecide-data/lean"
echo "TYPESAFE_API_KEY=your_key" > ../.env          # or export TYPESAFE_API_KEY=...
../.venv/bin/python run_jev.py                     # 515 calls, ~$0.005, resumable (JEV_LIMIT=20 for a
test)
../.venv/bin/python score.py --models edge_zeroshot base_zeroshot edge_A0 edge_A3 jev
open results/RESULTS.md
```

B. Finish the bigger model (optional, about 30-60 min on the M4, or cheap on Modal)

```
../.venv/bin/python train.py --base knowledgator/gliclass-modern-base-v3.0 --recipe A3 --lr 2e-5 --out
base_A3
../.venv/bin/python eval_local.py --model models/base_A3 --name base_A3 --latency
../.venv/bin/python score.py
```

C. Rebuild or retrain from scratch

```
../.venv/bin/python build_data.py        # eval stays byte-identical: shasum -a 256 -c
data/eval_v0.1.sha256
../.venv/bin/python train.py --recipe A3 --out edge_A3   # ~13 min on M4
../.venv/bin/python train.py --recipe A0 --out edge_A0
../.venv/bin/python eval_local.py --model models/edge_A3 --name edge_A3 [--split dev]
```

D. Recommended next improvements

• Fix unseen-style negation: more negation classes in train (while keeping one held-out class for the claim); undecidable 'trap' pairs (NLI neutral / CondaQA 'don't know') so 1-p shortcuts are penalised; try modern-base 151M; consider a per-type temperature fitted on dev.

• Add fresh, never-published eval items (removes 'Jev saw the public test sets' doubts). Add the AbdelStark Banking77 replay as a harness check that Jev reproduces about 87%.

• Report per the pre-registered framing: win on injection robustness, local latency and fine-tunability; state honestly where Jev wins (zero-shot breadth).

• If you pivot to tariff codes: CROSS rulings on HF (flexifyai/cross_rulings_hts_dataset_for_tariffs, about 18.7k rows), HSCodeComp and ATLAS; first check shortlist recall@64 with GTE-ModernBERT; kill the idea if it is below 90%.

8. File map (absolute paths)

Path	What
/Users/aryamahadik/Documents/551 CONTAINERS /ds-551-f26-c03-containers-aryamahadik2003/reports/JEV model and research startup ideas.md	Deep-research report (Jev, open models, $30 compute, funding, 6 ideas + dropped list), with a 25-Sep update
/Users/aryamahadik/Documents/551 CONTAINERS /ds-551-f26-c03-containers-aryamahadik2003/research_notes/JEV model and research startup ideas	Raw research notes incl. codex_claims_verification.md
/Users/aryamahadik/Documents/551 CONTAINERS /ds-551-f26-c03-containers-aryamahadik2003/opendecide-data/DESIGN.md	Full research-grade dataset spec (v1.0-rc) + build deviations + review log
/Users/aryamahadik/Documents/551 CONTAINERS /ds-551-f26-c03-containers-aryamahadik2003/opendecide-data/design	Scout notes (datasets, licences, formats, audits), 3 design proposals, design_plan.py
/Users/aryamahadik/Documents/551 CONTAINERS /ds-551-f26-c03-containers-aryamahadik2003/opendecide-data/lean	The lean benchmark that actually ran: code, data, models, preds, results
/Users/aryamahadik/Documents/551 CONTAINERS /ds-551-f26-c03-containers-aryamahadik2003/opendecide-data/assets/labels/keys	Natural-language label names for all datasets (used by lean)
/Users/aryamahadik/Documents/551 CONTAINERS /ds-551-f26-c03-containers-aryamahadik2003/opendecide-data/build	Partial full-spec pipeline (env, lock, banking77 loader); not needed for lean

Caveats: the eval is small (295 core groups; per-dataset CIs are wide). There are no trap pairs yet. The public test sets may be in Jev's training data. Propositions come from label names, not hand-written descriptions. Licences: train uses commercial-friendly sources (DBpedia, MNLI and wikitext are share-alike); SMS/TFNS are eval-only.